Aula 04 - Program Comprehension e Métricas de Complexidade

Disciplina: Manutenção de Software

Prof. Me. João Paulo Biazotto

UniCesumar

Compreensão de programas

Atividades para entender o software existente

Mudanças:

- Corretiva
- Preventiva
- Evolutiva
- Adaptativa

Compreensão de programas

Antes de implementar qualquer mudança é necessário:

- Conhecer o software como um todo
- Conhecer os detalhes relacionados à mudança

Compreensão de programas - Etapas

- Conhecimento geral sobre o que o software faz
- Identificar onde as alterações devem ser realizadas
- Conhecimento mais profundo sobre como as partes a serem alteradas funcionam

Compreensão de programas - Etapas

- Compreensão de programas consome uma proporção significativa de esforço de manutenção

- Dados podem variar de acordo com a qualidade do código, documentação, nível de experiência do desenvolvedor, tipo da correção, etc

Compreensão de programas - Etapas

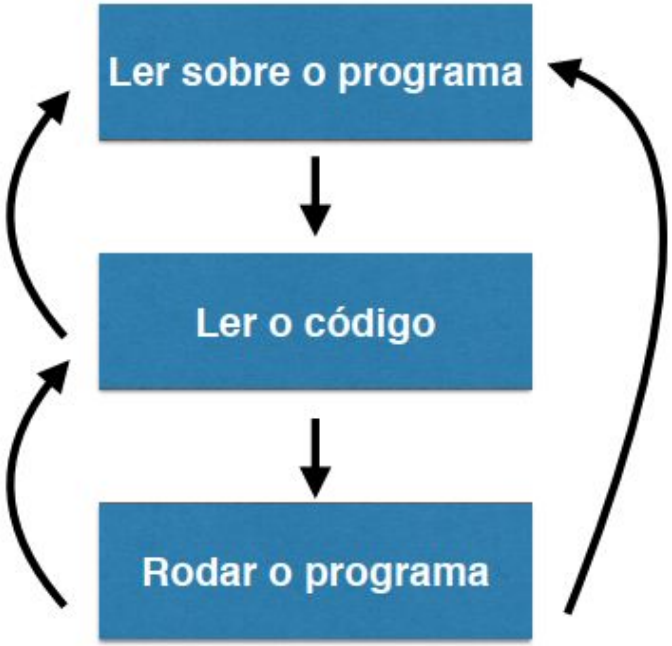
- Isso envolve lidar com certos aspectos:
- Domínio do problema (para estimar esforço)
  - Efeito de execução (verificar se mudança teve efeito desejado)
  - Relação causa-efeito (análise de impacto)
  - Ferramentas de suporte (métricas de complexidade, manutenabilidade, engenharia reversa)

Modelo de Compreensão

De modo geral são três tarefas:

- **Ler sobre o programa**: acessar documentação (especificação, projeto) se disponível
- **Ler o código:** obter uma visão global e local do programa, usar ferramentas de engenharia reversa
- **Rodar o programa:** revelar características do sistema difíceis de obter apenas lendo o código

Modelo de Compreensão



Fatores que podem afetar a compreensão de programas

- O que torna um programa mais difícil de entender?

Fatores que podem afetar a compreensão de programas

- Formatação do código

```
<?php
class Calculator {
public function add($a, $b) { return $a + $b; }
public function multiply($a, $b) { return $a * $b;
} public function divide($a, $b) { if($b == null) {
throw new Exception("Division by zero"); } return $a / $b; }
public function subtract($a, $b) { return $a - $b; } }
?>
```

Fatores que podem afetar a compreensão de programas

- Formatação do código

```php
<?php
class Calculator {
    public function add($a, $b) {
        return $a + $b;
    }
    public function multiply($a, $b) {
        return $a * $b;
    }
    public function divide($a, $b) {
        if ($b == null) {
            throw new Exception ( "Division by zero" );
        }
        return $a / $b;
    }
    public function subtract($a, $b) {
        return $a - $b;
    }
}
?>
```

Fatores que podem afetar a compreensão de programas

- Conhecimento sobre o domínio
- Suporte de ferramentas de engenharia reversa
- Qualidade do código (legibilidade, complexidade, uso de padrões de projeto, etc)
- Qualidade da documentação (interna e externa)
- Nível de experiência do desenvolvedor

Fatores que podem afetar a compreensão de programas

- **Conhecimento sobre o domínio**
  - Ao longo do tempo, desenvolvedores tornam-se especialistas em um determinado domínio (LP, BD, bibliotecas, etc)

Fatores que podem afetar a compreensão de programas

- **Conhecimento sobre o domínio**
  - Logo, o nível de expertise tem um papel significante na compreensão de programas
  - Estudos mostram que o nível de expertise influencia positivamente no entendimento

Fatores que podem afetar a compreensão de programas

- **Suporte de ferramentas de engenharia reversa**
- Exemplos:
  - Moose, Jarchitect, SonarQube, MaintainJ, Findbugs, Lint, Newrelic, IDA Pro, Hiew, Scylla

Fatores que podem afetar a compreensão de programas

- **Qualidade do código (legibilidade, complexidade, uso de padrões de projeto, etc)**
  - Complexidade
  - Legibilidade
  - Comentários em código
  - Formatação

Qualidade do código - Complexidade

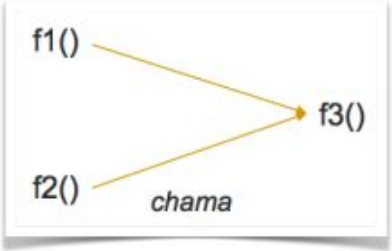
- Exemplos de métricas:
  - Fan-in
  - Fan-out
  - Complexidade ciclomática
  - Tamanho do código
  - Profundidade de aninhamento
  - Tamanho do vocabulário

Qualidade do código - Complexidade

- Exemplos de métricas:
  - **Fan-in**
  - **Fan-out**
  - **Complexidade ciclomática**
  - Tamanho do código
  - Profundidade de aninhamento
  - Tamanho do vocabulário

Fan-in

- Número de funções que chamam uma dada função
- Valor alto significa grande impacto em mudanças (propagação)



fan-in(f3) = 2

Fan-out

- Número de funções chamadas pela função
- Valor alto significa grande complexidade da função

Fan-in e Fan-out

- Exemplo
- Suponha que temos os seguintes módulos:
- - Módulo A: chamado pelos módulos B, C e D.
- - Módulo B: chama os módulos E e F.
- - Módulo C: chama os módulos G e H.
- **Calcule Fan-in de A e Fan-out de B e C**

Fan-in e Fan-out

- Calcule Fan-in de A e Fan-out de B e C
  - Fan-in de A: 3 (B, C, D)
  - Fan-out de B: 2 (E, F)
  - Fan-out de C: 2 (G, H)

Fan-in e Fan-out

- Dado o seguinte sistema:
  - Módulo X: chamado por A, B.
  - Módulo Y: chamado por X.
  - Módulo Z: chamado por X, Y.
  - Módulo W: chama X, Y, Z.

- - Calcule o **Fan-in** e o **Fan-out** para os módulos X, Y, Z, e W.

Fan-in e Fan-out

- Fan-in de X: 3 (A, B, W)
- Fan-out de X: 2 (Y, Z)
- Fan-in de Y: 2 (X, W)
- Fan-out de Y: 1 (Z)
- Fan-in de Z: 3 (X, Y, W)
- Fan-out de Z: 0
- Fan-in de W: 0
- Fan-out de W: 3 (X, Y, Z)

Complexidade ciclomática

- Indica a complexidade do código através da quantidade de caminhos de execução
- Mede a complexidade de controle do programa
  - if, while, for, etc
- Relacionada à facilidade de compreensão

Complexidade ciclomática

- Mede a complexidade de um programa com base no número de caminhos lineares independentes pelo código.
- É calculada usando o gráfico de fluxo de controle do programa, como M = E - N + 2P,
- Onde E é o número de arestas, N é o número de nós, e P é o número de componentes conectados.

Complexidade ciclomática

```
int CalcularDesconto(int valor) {
    if (valor > 5000) return 20;
    if (valor > 3000) return 15;
    if (valor > 1000) return 10;
    return 0;
}
```

Complexidade ciclomática

O grafo de fluxo de controle para este código seria:

- Nó 1: Início do método.
- Nó 2: Condição valor > 5000.
- Nó 3: Condição valor > 3000.
- Nó 4: Condição valor > 1000.
- Nó 5: return 0.

Complexidade ciclomática

- Nó 1: Início do método.
- Nó 2: Verificação if (valor > 5000).
- Se verdadeiro, vai para return 20.
- Se falso, vai para a próxima verificação.
- Nó 3: Verificação if (valor > 3000).
- Se verdadeiro, vai para return 15.
- Se falso, vai para a próxima verificação.

Complexidade ciclomática

- Nó 4: Verificação if (valor > 1000).
- Se verdadeiro, vai para return 10.
- Se falso, vai para return 0.
- Nó 5: Retorno final return 0.

Complexidade ciclomática

Calcule os Valores

- E (arestas): Transições entre os nós (7 no total).
- N (nós): 5 (cada bloco de código e decisão).
- P (componentes): 1 (é um único método).

Complexidade ciclomática

Aplicando a fórmula:
Complexidade Ciclomática = 7 – 5 + 2 (1) = 4

- A complexidade ciclomática é 4, o que significa que há quatro caminhos independentes que você precisaria testar para cobrir todas as condições do método. Um valor mais alto indicaria maior complexidade e mais esforço de teste.

DÚVIDAS?